

CENTRO UNIVERSITÁRIO ESTÁCIO DO RECIFE
Campus Abdias de Carvalho

Ponte

“A solução está do outro lado”

Anna Beatriz Rodrigues de Moura

David Arthur de Santana Albuquerque

Ezequiel Barbosa da Silva

Matheus Nunes da Silva

Nicolas Emanuel de Sena Cajueiro

ORIENTADOR: Davi Camara

2026
Recife/PE

Sumário

1.	DIAGNÓSTICO E TEORIZAÇÃO	4
1.1.	Identificação das partes interessadas e parceiros	4
1.2.	Problemática e/ou problemas identificados	4
1.3.	Justificativa	5
1.4.	Objetivos/resultados/efeitos a serem alcançados (em relação ao problema identificado e sob a perspectiva dos públicos envolvidos)	5
1.5.	Referencial teórico (subsídio teórico para proposição de ações da extensão)	6
2.	PLANEJAMENTO E DESENVOLVIMENTO DO PROJETO	6
2.1.	Plano de trabalho (usando ferramenta acordada com o docente)	7
2.2.	Grupo de trabalho (descrição da responsabilidade de cada membro)	8
2.3.	Metas, critérios ou indicadores de avaliação do projeto	8
2.4.	Recursos previstos	9
2.5.	Detalhamento técnico do projeto.....	10
3.	ENCERRAMENTO DO PROJETO.....	11
3.1.	Relato Coletivo	11
3.2.	Relato de Experiência Individual (Pontuação específica para o relato individual)	12
3.2.1.	CONTEXTUALIZAÇÃO	12
3.2.2.	METODOLOGIA	13
3.2.3.	RESULTADOS E DISCUSSÃO.....	13
3.2.4.	REFLEXÃO APROFUNDADA.....	14
3.2.5.	CONSIDERAÇÕES FINAIS	14

1. DIAGNÓSTICO E TEORIZAÇÃO

1.1. Identificação das partes interessadas e parceiros

O projeto Ponte foi concebido a partir da observação das necessidades de comunicação interna em ambientes corporativos, especialmente no que diz respeito ao relacionamento entre colaboradores e o Departamento de TI. As partes interessadas diretamente envolvidas no projeto são:

- Colaboradores comuns da empresa: usuários finais do sistema, responsáveis por registrar e acompanhar os chamados técnicos abertos junto ao setor de TI.
- Equipe de TI (Administradores): profissionais dos departamentos de Hardware, Software/Sistema, Redes e Desenvolvimento, responsáveis por receber, triar, encaminhar e resolver os chamados registrados.
- Gestores e líderes organizacionais: beneficiários indiretos do sistema, uma vez que a otimização dos processos de suporte impacta diretamente a produtividade das equipes.
- Equipe de desenvolvimento (Grupo acadêmico): Ezequiel, Anna, David, Matheus e Nicolas, estudantes da disciplina de Programação para Dispositivos Móveis em Android, responsáveis pelo desenvolvimento completo do sistema.
- Docente orientador: Davi Camara, acompanha institucionalmente o projeto, orienta as decisões técnicas e acadêmicas e valida as entregas.

1.2. Problemática e/ou problemas identificados

Em muitas organizações, a comunicação entre os colaboradores e o setor de TI ocorre de forma informal e descentralizada — por meio de mensagens instantâneas, ligações telefônicas, e-mails sem padronização ou até mesmo de forma verbal. Esse cenário gera uma série de problemas críticos:

- Falta de rastreabilidade dos chamados: sem um sistema centralizado, é difícil acompanhar o status de uma solicitação ou identificar gargalos no atendimento.
- Encaminhamento incorreto de demandas: sem categorização estruturada, chamados frequentemente chegam ao setor errado, gerando retrabalho e atraso na resolução.
- Comunicação ineficiente: o colaborador muitas vezes não sabe se o seu problema está sendo tratado, gerando ansiedade, repetição de contatos e sobrecarga da equipe de TI.
- Ausência de histórico: sem registro estruturado, torna-se impossível analisar padrões de ocorrências ou medir o desempenho do setor de suporte.

- Impacto no bem-estar e na produtividade: a demora na resolução de problemas técnicos afeta diretamente o ambiente de trabalho e a satisfação dos colaboradores.

Esses problemas evidenciam a necessidade de uma ferramenta digital estruturada que centralize, organize e agilize o fluxo de comunicação entre colaboradores e a equipe de TI.

1.3. Justificativa

A criação do Ponte justifica-se pela necessidade identificada de modernizar e estruturar os processos de suporte técnico interno em ambientes corporativos. A adoção de sistemas digitais de gerenciamento de chamados (conhecidos como sistemas de ticketing) é amplamente reconhecida como uma prática eficaz para elevar a qualidade do atendimento técnico e a satisfação dos usuários.

Do ponto de vista acadêmico, o projeto representa uma oportunidade concreta de aplicar os conhecimentos adquiridos na disciplina de Programação para Dispositivos Móveis em Android, integrando tecnologias modernas como Ionic, Angular e Spring Boot na construção de uma solução fullstack funcional e implantada em produção.

A escolha de priorizar a experiência do usuário (UX/UI) no design do aplicativo reforça o compromisso do grupo com a usabilidade e a acessibilidade da ferramenta, tornando-a intuitiva tanto para colaboradores com menor familiaridade tecnológica quanto para profissionais experientes da área de TI. O nome Ponte reflete exatamente esse propósito: conectar o problema à sua solução de forma ágil, transparente e humanizada.

1.4. Objetivos/resultados/efeitos a serem alcançados (em relação ao problema identificado e sob a perspectiva dos públicos envolvidos)

Desenvolver o sistema Ponte, uma aplicação web e mobile capaz de centralizar a abertura e o acompanhamento de chamados técnicos internos, permitindo que qualquer colaborador com acesso a computador, tablet ou celular registre ocorrências de forma estruturada — seja um problema de hardware, falha em software, indisponibilidade de rede ou qualquer outra demanda de suporte — eliminando a dependência de processos manuais, informais ou descentralizados para o reporte de problemas.

Implementar um fluxo organizado de triagem e encaminhamento de chamados, no qual cada ocorrência registrada é categorizada e direcionada

automaticamente à equipe responsável (infraestrutura, desenvolvimento, redes, entre outras), otimizando o tempo de resposta, reduzindo retrabalho por falta de informação e oferecendo rastreabilidade completa do status de cada chamado para o colaborador que o abriu.

Para demonstrar os resultados alcançados, serão utilizados testes funcionais com usuários representando diferentes perfis de colaboradores, avaliando a facilidade de abertura de chamados, a clareza das categorias disponíveis, a correção do encaminhamento por tipo de problema e a percepção geral de melhoria em relação ao processo anterior.

1.5. Referencial teórico (subsídio teórico para propositura de ações da extensão)

O desenvolvimento do Ponte fundamenta-se em conceitos consolidados das áreas de Engenharia de Software, Gestão de Serviços de TI e Design de Interação.

No campo da Gestão de Serviços de TI, o projeto se alinha aos princípios da biblioteca ITIL (Information Technology Infrastructure Library), especialmente no que tange ao processo de Gerenciamento de Incidentes, que preconiza o registro estruturado, categorização, priorização e escalonamento de chamados técnicos como meio de minimizar o impacto de interrupções no ambiente de trabalho.

No âmbito do Design de Interação e Experiência do Usuário (UX/UI), o projeto adota princípios estabelecidos por Jakob Nielsen (2000), especialmente as heurísticas de usabilidade, que incluem visibilidade do status do sistema, correspondência entre o sistema e o mundo real, controle e liberdade do usuário, consistência e padrões, e design minimalista. Esses princípios orientaram as escolhas de interface, resultando em um app com cores leves, navegação intuitiva e botões didáticos.

Do ponto de vista tecnológico, a arquitetura desacoplada (frontend e backend independentes, comunicando-se via API REST) segue os princípios REST descritos por Roy Fielding (2000), favorecendo a escalabilidade, a manutenção e a interoperabilidade do sistema. A adoção de autenticação baseada em tokens JWT e controle de acesso por papéis (RBAC) alinha-se às boas práticas de segurança em aplicações web modernas.

2. PLANEJAMENTO E DESENVOLVIMENTO DO PROJETO

2.1. Plano de trabalho (usando ferramenta acordada com o docente)

O desenvolvimento do sistema Ponte foi organizado em etapas progressivas, cobrindo desde a concepção da arquitetura até a implantação em ambiente de produção. A ferramenta utilizada para gestão e acompanhamento das tarefas foi acordada com o docente orientador, garantindo rastreabilidade do progresso ao longo do projeto.

Etapas do Desenvolvimento:

Levantamento de Requisitos: Identificação das funcionalidades essenciais para substituição do sistema existente na empresa parceira, com mapeamento dos fluxos de criação, atribuição, acompanhamento e encerramento de chamados.

Modelagem e Arquitetura: Definição do modelo de domínio com entidades ricas, relacionamentos e regras de negócio; projeto das rotas da API REST; estruturação do banco de dados com migrações gerenciadas pelo Flyway.

Desenvolvimento do Backend: Implementação da API com Spring Boot, incluindo autenticação via JWT com Spring Security, mapeamento de entidades com JPA/Hibernate, DTOs com MapStruct e controle de acesso baseado em papéis (RBAC).

Desenvolvimento do Frontend: Construção das telas com Angular e Ionic utilizando Reactive Forms, integração com a API via serviços HTTP, controle de estado e exibição de dados em tempo real com operadores reativos (RxJS).

Containerização e Configuração de Ambiente: Criação do Dockerfile multi-stage para o backend Spring Boot; configuração do ambiente de banco de dados no Supabase; definição de variáveis de ambiente para separação de configurações sensíveis.

Testes: Realização de testes unitários com JUnit 5 e Mockito nas camadas de serviço, e testes de integração com MockMvc para validação dos endpoints da API.

Deploy e Validação: Publicação do backend no Render com integração contínua via repositório Git; validação dos fluxos completos em ambiente de produção; ajustes de CORS, autenticação e performance.

2.2. Grupo de trabalho (descrição da responsabilidade de cada membro)

A equipe de desenvolvimento do sistema Ponte é composta por cinco integrantes, com funções distribuídas de acordo com as necessidades do projeto e as habilidades de cada membro:

Matheus atuou como desenvolvedor frontend. Foi responsável por converter os protótipos de design da plataforma em código funcional, além de contribuir na implementação da lógica de comportamento, dinamismo e interatividade da página de interesses.

David atuou no desenvolvimento frontend com foco especial em UX/UI & Design, sendo responsável pela implementação de recursos visuais que aprimoram a experiência do usuário, como o skeleton loading nos cards durante o carregamento de dados.

Anna foi responsável pelo desenvolvimento backend, implementando a lógica de negócio, modelagem de dados, endpoints da API e operações CRUD do usuário, além da documentação do projeto, relatório e slide.

Ezequiel atuou no desenvolvimento frontend, sendo também responsável pelo design de telas e experiência do usuário (UX/UI), além da elaboração da documentação do projeto, incluindo relatório e materiais de apresentação.

Nicolas desempenhou o papel de desenvolvedor fullstack, transitando entre as camadas do sistema, e ficou responsável pela infraestrutura e DevOps, cuidando da containerização com Docker, configuração de ambientes e deploy da aplicação em produção.

2.3. Metas, critérios ou indicadores de avaliação do projeto

O sucesso do Ponte será avaliado com base nos seguintes critérios e indicadores:

- Funcionalidade completa do fluxo de chamados: o sistema deve permitir abertura, categorização, acompanhamento e encerramento de chamados sem falhas críticas.
- Autenticação e controle de acesso: o sistema deve garantir separação clara de permissões entre perfil de colaborador comum e administrador.
- Encaminhamento correto de chamados: chamados devem ser direcionados automaticamente ao departamento correto (Hardware, Software/Sistema, Redes ou Desenvolvimento).
- Usabilidade percebida: avaliada por meio de testes funcionais com usuários, mensurando facilidade de uso, clareza das opções e satisfação geral.
- Disponibilidade em produção: o sistema deve estar disponível via web e como aplicativo Android instalado em dispositivo real.
- Qualidade do código: avaliada pela cobertura de testes unitários e de integração implementados com JUnit 5, Mockito e MockMvc.

- Comunicação em tempo real: as atualizações enviadas pelo administrador no chat devem ser refletidas imediatamente na interface do colaborador.

2.4. Recursos previstos

Para a execução do projeto, foram utilizados recursos materiais, institucionais e humanos que não geraram custos adicionais. Entre os recursos materiais, utilizamos computadores pessoais dos integrantes, acesso à internet para pesquisas e desenvolvimento.

No âmbito institucional, contamos com o suporte da instituição de ensino, incluindo acesso a salas de estudo, laboratórios equipados, ambiente virtual de aprendizagem

Recursos Tecnológicos

Tecnologia	Finalidade	Licença/Plano
Angular + Ionic	Desenvolvimento do frontend web e mobile	Open source
Capacitor	Geração do app Android	Open source
Spring Boot	Desenvolvimento da API REST	Open source
Docker	Containerização do backend	Free
PostgreSQL	Banco de dados relacional	Open source
Supabase	Hospedagem gerenciada do banco de dados	Free tier
Render	Deploy do backend containerizado	Free tier
Vercel / Netlify	Deploy do frontend Angular	Free tier
Git / GitHub	Versionamento e colaboração	Free

2.5. Detalhamento técnico do projeto

Visão Geral da Arquitetura

O Ponte é um sistema de gerenciamento de chamados (tickets) desenvolvido como uma aplicação fullstack desacoplada. O frontend e o backend se comunicam exclusivamente via API REST com autenticação baseada em tokens JWT, sem compartilhamento de estado entre as camadas.

Frontend

O frontend é desenvolvido com Angular utilizando o framework Ionic, que fornece componentes de interface adaptados para dispositivos móveis e desktop. A aplicação é estruturada em módulos e utiliza Reactive Forms para gerenciamento de formulários com validação reativa. A comunicação com a API é feita por meio de serviços Angular com o HttpClient, e o fluxo assíncrono de dados é gerenciado com RxJS, utilizando operadores como switchMap para dependências entre requisições.

A aplicação é compilada e empacotada pelo Capacitor, que converte o projeto web em um aplicativo Android nativo, já testado em dispositivo. O deploy da versão web é realizado em plataforma de hospedagem estática (Vercel ou Netlify), com build gerado pelo Angular CLI.

Backend

O backend é uma API REST desenvolvida com Spring Boot, seguindo princípios de modelo de domínio rico, onde as entidades encapsulam regras de negócio. Os principais componentes técnicos incluem:

- Spring Security com JWT: autenticação stateless com geração e validação de tokens. O controle de acesso é baseado em roles (papéis).
- JPA/Hibernate: mapeamento objeto-relacional das entidades, com suporte a consultas JPQL e nativas.
- MapStruct: mapeamento entre entidades de domínio e DTOs, evitando exposição direta das entidades nas respostas da API.
- Flyway: gerenciamento de migrações do banco de dados com versionamento, garantindo consistência do schema entre ambientes.

- Docker: a aplicação Spring Boot é empacotada em uma imagem Docker via Dockerfile multi-stage, reduzindo o tamanho final da imagem.

Banco de Dados

O banco de dados utilizado é o PostgreSQL, provisionado e gerenciado pelo Supabase. Essa escolha oferece uma instância PostgreSQL totalmente gerenciada com painel de administração. O schema é versionado e aplicado automaticamente pelo Flyway na inicialização do backend.

Deploy e Infraestrutura

O backend containerizado é implantado na plataforma Render, que detecta automaticamente o Dockerfile no repositório e realiza o build e deploy a cada push na branch principal. As variáveis de ambiente sensíveis (credenciais do banco, chave JWT, etc.) são configuradas diretamente no painel do Render. O frontend é implantado separadamente em plataforma de hospedagem estática.

Segurança

- Senhas armazenadas com hash BCrypt.
- Autenticação via JWT com tempo de expiração configurável.
- Controle de acesso por roles aplicado nos endpoints da API via Spring Security.
- Variáveis sensíveis isoladas em variáveis de ambiente, fora do código-fonte.

3. ENCERRAMENTO DO PROJETO

3.1. Relato Coletivo:

O desenvolvimento do Ponte representou uma experiência completa para todos os integrantes do grupo. Ao longo das etapas do projeto, foi possível vivenciar na prática os desafios reais de construção de uma aplicação fullstack, desde a concepção da ideia até o deploy em produção.

A escolha do nome Ponte nasceu de uma reflexão profunda sobre o papel do sistema: mais do que um gerenciador de chamados, a ferramenta é uma ponte entre o problema do colaborador e a solução oferecida pelo time de TI. Essa mentalidade orientou cada decisão de design e desenvolvimento, priorizando sempre a experiência do usuário final.

No aspecto técnico, o grupo enfrentou desafios significativos, especialmente na integração entre o frontend Angular/Ionic e o backend Spring Boot, na configuração correta do CORS em ambiente de produção e na geração do aplicativo Android via Capacitor. Cada obstáculo foi superado por meio de pesquisa, colaboração e persistência, reforçando o valor do trabalho em equipe.

A pesquisa de usabilidade realizada antes e durante o desenvolvimento foi fundamental para garantir que o produto fosse intuitivo e acessível. A aplicação de conceitos de UX/UI, como o uso de cores leves, botões didáticos e navegação dinâmica, resultou em uma interface que foi bem avaliada nos testes com usuários.

Do ponto de vista acadêmico, o projeto consolidou os conhecimentos da disciplina de Programação para Dispositivos Móveis em Android e abriu portas para aprendizados complementares em áreas como DevOps (Docker, Render), segurança de aplicações (JWT, BCrypt) e banco de dados em nuvem (Supabase/PostgreSQL).

O grupo avalia o Ponte como um projeto bem-sucedido, não apenas pelos resultados técnicos alcançados, mas pelo impacto positivo que uma ferramenta como essa pode gerar no cotidiano de uma organização. A experiência reforçou a importância de desenvolver soluções centradas nas pessoas, transformando dores reais em tecnologia útil e acessível.

3.2. Relato de Experiência Individual (Pontuação específica para o relato individual)

3.2.1. CONTEXTUALIZAÇÃO

Eu, Nicolas Emanuel de Sena Cajueiro, participei do projeto Ponte. Que surgiu como proposta de substituição de um sistema existente de abertura de chamados técnicos internos de uma empresa, com o objetivo de centralizar e organizar o fluxo de suporte entre colaboradores e equipes responsáveis. A equipe foi composta por cinco desenvolvedores, cada um com responsabilidades definidas dentro da arquitetura do sistema.

Minha participação no projeto se deu como desenvolvedor fullstack, acumulando também as responsabilidades de definição da arquitetura do sistema e configuração do ambiente de produção. Partindo de experiência prévia com as tecnologias adotadas, coube a mim tomar as decisões estruturais do projeto: a escolha do stack tecnológico, a organização das camadas da aplicação, os padrões de comunicação entre frontend e backend, e a estratégia de deploy adotada.

No backend, atuei na implementação da lógica de negócio com Spring Boot, incluindo a estruturação das regras que regem o ciclo de vida dos chamados e a configuração da autenticação com JWT. No frontend, desenvolvi partes da aplicação Angular, contribuindo especialmente na integração com a API e na implementação de lógicas de negócio no lado do cliente. Na infraestrutura, fui responsável pelo processo completo de deploy: configuração da aplicação backend no Render, provisionamento do banco de dados no Supabase e definição das variáveis de ambiente necessárias para o funcionamento seguro do sistema em produção.

Essa experiência consolidou na prática o ciclo completo de desenvolvimento de um sistema — da concepção da arquitetura à entrega em ambiente real — e reforçou a importância de decisões técnicas bem fundamentadas desde as fases iniciais do projeto.

3.2.2. METODOLOGIA

A experiência foi realizada em diferentes ambientes ao longo do projeto, incluindo sala de aula, laboratório e encontros por chamadas online, que facilitaram a comunicação e o andamento das atividades mesmo fora do espaço físico da instituição. Grande parte do desenvolvimento ocorreu de forma remota, com a equipe se organizando por meio de ferramentas de comunicação à distância.

Os sujeitos envolvidos no desenvolvimento do projeto foram os membros da equipe: Matheus, Nicolas, Ezequiel, David e eu, além do professor orientador Davi Camara.

O desenvolvimento do projeto aconteceu por etapas. Primeiro, realizamos o levantamento do problema e compreendemos a necessidade principal do trabalho. Em seguida, fizemos o planejamento e definimos a divisão das tarefas, para que cada integrante soubesse exatamente sua função.

Na parte técnica, atuei como desenvolvedor fullstack, sendo responsável tanto pelo backend quanto por partes do frontend da aplicação, além da definição da arquitetura do sistema e da configuração do ambiente de produção. No backend, utilizei Spring Boot para a construção da API REST, incluindo a lógica de negócio e a autenticação dos usuários. No frontend, contribuí com a integração entre a interface Angular e a API. Também fui responsável pelo deploy completo da aplicação, configurando o ambiente de produção no Render, o banco de dados no Supabase e as variáveis de ambiente necessárias para o funcionamento seguro do sistema.

Durante esse período, também foram realizadas reuniões periódicas para alinhar ideias, revisar o andamento e corrigir possíveis falhas. Por fim, concluímos o projeto com a apresentação dos resultados.

3.2.3. RESULTADOS E DISCUSSÃO:

A expectativa inicial era desenvolver um sistema funcional de gerenciamento de chamados que pudesse ser utilizado em um contexto real, e essa expectativa foi plenamente atendida. Todas as funcionalidades previstas foram implementadas e entregues, com o sistema operando em produção e acessível via navegador e

dispositivo Android.

A integração entre o frontend Angular e a API REST foi o ponto que fluiu com mais naturalidade, beneficiada pela experiência prévia com as tecnologias utilizadas. A principal dificuldade esteve no frontend, especificamente na combinação de Angular com Ionic, que apresenta particularidades no comportamento de componentes e formulários que exigiram investigação e atenção redobrada.

A colaboração entre os cinco integrantes foi equilibrada e positiva ao longo de todo o projeto. Uma das principais aprendizagens foi vivenciar o ciclo completo de uma aplicação em produção, reforçando que decisões de arquitetura tomadas no início impactam diretamente a facilidade do deploy e da manutenção.

Como recomendação, sugere-se que projetos futuros com stack semelhante dediquem tempo específico ao estudo das particularidades do Ionic com Angular standalone desde o início, e que a configuração do ambiente de produção seja tratada como etapa do desenvolvimento, e não como tarefa final.

3.2.4. REFLEXÃO APROFUNDADA

A disciplina de Programação para Dispositivos Móveis apresentou os fundamentos do desenvolvimento mobile utilizando Ionic e TypeScript como base tecnológica, com Angular como framework principal para a construção das interfaces. Durante as aulas, o conteúdo foi abordado de forma introdutória, cobrindo a estrutura de componentes, navegação e o uso do TypeScript no ecossistema Angular/Ionic.

No desenvolvimento do Ponte, esse conteúdo foi colocado em prática em um contexto real e de maior complexidade do que o abordado em sala. A teoria forneceu a base necessária para compreender a estrutura do framework, porém a experiência vivida no projeto evidenciou aspectos que vão além do que é possível cobrir em ambiente acadêmico — como o comportamento específico dos componentes Ionic em modo standalone, a integração com Reactive Forms e a comunicação assíncrona com uma API REST via RxJS.

Essa diferença entre o conteúdo teórico e a aplicação prática foi um dos aprendizados mais significativos do projeto: a teoria orienta e estrutura o raciocínio, mas é na prática — com seus erros, investigações e ajustes — que o conhecimento se consolida de fato. O Ponte representou, nesse sentido, uma extensão natural da disciplina, exigindo que o conteúdo aprendido fosse adaptado, aprofundado e aplicado sob as demandas reais de um sistema em produção.

3.2.5. CONSIDERAÇÕES FINAIS

O desenvolvimento do Ponte atingiu os objetivos propostos, resultando em um sistema funcional de gerenciamento de chamados acessível via web e dispositivo Android. Ainda assim, há aspectos que podem ser evoluídos em trabalhos futuros.

Perspectivas de evolução do sistema

Um caminho natural de expansão seria a implementação de notificações em tempo real, alertando o colaborador sobre atualizações no status do seu chamado sem necessidade de recarregar a tela. Tecnologias como WebSocket ou Server-Sent Events integradas ao backend Spring Boot viabilizariam essa funcionalidade. Outro ponto de melhoria seria a criação de um painel administrativo com métricas e relatórios — tempo médio de resolução por categoria, volume de chamados por setor e indicadores de desempenho das equipes de suporte — agregando valor gerencial ao sistema.

A publicação do aplicativo Android nas lojas (Google Play) também representa uma perspectiva concreta de trabalho futuro, tornando o acesso mais

prático para os colaboradores sem necessidade de instalação manual via APK.

Soluções tecnológicas alternativas

O mesmo sistema poderia ter sido desenvolvido com tecnologias distintas. No backend, frameworks como Node.js com NestJS ou Python com FastAPI seriam alternativas viáveis, com curva de aprendizado possivelmente menor para parte da equipe. No frontend, React Native substituiria a combinação Angular/Ionic/Capacitor para o desenvolvimento mobile, oferecendo uma experiência mais próxima do nativo. Para o banco de dados, Firebase poderia ter sido adotado como alternativa ao PostgreSQL com Supabase, simplificando a configuração inicial em troca de menor controle sobre a estrutura dos dados.

A escolha pelo stack adotado — Spring Boot, Angular, Ionic e PostgreSQL — foi motivada pela experiência prévia da equipe e pela robustez das tecnologias para um sistema de uso corporativo, justificando a decisão mesmo diante das alternativas existentes.